

Parallel $\mathcal{O}(\sqrt{n})$ Overhead LSD Radix Sort

Robert Clausecker ✉

Zuse Institute Berlin, Germany

Florian Schintke ✉ 

Zuse Institute Berlin, Germany

Abstract

We present Radsort, a variant of LSD radix sort, sorting data with $\mathcal{O}(\sqrt{n})$ additional space. Radsort is stable, admits a simple implementation and is easy to parallelise. For arrays exceeding a size of around 2 MiB it outperforms a conventional out-of-place LSD radix sort.

2012 ACM Subject Classification Theory of computation → Sorting and searching

Keywords and phrases cache locality, radix sort, sorting

Supplementary Material <https://github.com/clausecker/radsort>

Funding *Robert Clausecker*: NHR-Verein Graduate School programme

Acknowledgements This work was produced during a secondment of the first author at the Sanders workgroup at KIT. The author would like to thank Marvin Williams for his invaluable input.

1 Introduction

An LSD (least significant digit) radix sort algorithm sorts an array A of length n holding n_t -tuples of keys $(key(A[i], 0), key(A[i], 1), \dots, key(A[i], n_t - 1))$ of alphabet Σ with σ symbols into lexicographic order by repeatedly sorting according to $key(A[i], n_t - 1)$, then $key(A[i], n_t - 2)$, and so on until $key(A[i], 0)$. With a stable sort, identical keys end up sorted by their suffixes, giving a lexicographic order. A conventional LSD radix sort sorts by $key(A[i], t)$ by first taking a histogram H of how often each $c \in \Sigma$ occurs. A prefix sum over H obtains the bucket starts S , with which we sort A into a new array A' (Alg. 1).

Algorithm 1 out-of-place LSD radix sort round

```
1 procedure RADIXSORTSTEP( $A', A, t$ )                                ▷ sort array  $A$  by key  $t$  into  $A'$ 
2    $H \leftarrow 0 \dots 0$ 
3   for  $i \leftarrow 0 \dots n - 1$  do                                    ▷ take a histogram of  $key(A[i], t)$ 
4      $H[key(A[i], t)] \leftarrow H[key(A[i], t)] + 1$ 
5   end for
6    $S[0] \leftarrow 0$ 
7   for  $i \leftarrow 1 \dots \sigma - 1$  do
8      $S[i] \leftarrow S[i - 1] + H[i - 1]$                                 ▷ prefix sum over  $H$ 
9   end for
10  for  $i \leftarrow 0 \dots n - 1$  do
11     $c \leftarrow key(A[i], t)$                                           ▷ determine bucket
12     $A'[S[c]] \leftarrow A[i]$                                           ▷ sort  $A[i]$  into  $A'$ 
13     $S[c] \leftarrow S[c] + 1$                                           ▷ advance bucket  $c$  past new element
14  end for
15 end procedure
```

With the input array A being consumed, output is produced in a separate array A' . While only n elements are occupied in both arrays together at any point in the algorithm, nevertheless $2n$ elements of storage, i.e. n elements of overhead in addition to the input, need to be provided. We would like to address this shortcoming with a novel LSD radix sort algorithm that does the job with only $\mathcal{O}(\sqrt{n})$ extra space at a similar performance. We call this algorithm *Radsort* due to it being a **radical** overhead **radix** sort.

■ **Algorithm 2** the Radsort algorithm

```

1 procedure RADSORT( $A, n, n_t$ )
2   SETUP( $A, n$ ) ▷ Alg. 4
3   for  $t \leftarrow n_t - 1 \dots 0$  do
4     SORTPHASE( $t$ ) ▷ Alg. 5
5     FIXUPPHASE ▷ Alg. 6
6   end for
7   FINALIZE ▷ Alg. 7
8 end procedure

```

2 Radsort

Radsort (Alg. 2) treats the input array A as a sequence of *blocks* of some block size b^1 . After a block of input is consumed, its storage is reused for subsequently produced output, reducing the storage overhead to a constant number of *scratch blocks*, as well as some bookkeeping.

Each round of sorting comprises two *phases*. During the *sort phase*, one output block is initially assigned to each of the σ *buckets* we sort into. Once such a block is full, a new block is drawn from previously consumed input blocks, overwriting the input as it is processed. The result of the sort phase are σ randomly interleaved sequences of output blocks, each forming one of the output buckets. In the *fixup phase* following each *sort phase*, these sequences are deinterleaved, giving the data block in order of buckets. The deinterleaving does not move data blocks around—instead, a permutation π tracks the order in which the data blocks need to be processed for the data to appear in order.

Like in a conventional LSD radix sort, these two phases are repeated for each key position in turn. After all n_t rounds of sorting, a *finalisation step* shuffles the data blocks around, giving a sorted result in A . While Radsort allocates space in a more complicated way than a traditional LSD radix sort, it works according to the same principles, giving the same stability guarantees at lower memory overhead and better cache locality.

2.1 Data Structures

In addition to the $\lfloor n/b \rfloor$ blocks of A , 2σ *scratch blocks* stored in the temporary array T are required: σ blocks of *head start* to cover the initial output blocks for the buckets of the current round, and σ additional covering the partial blocks of the previous round of sorting. The algorithm treats the concatenation of T and A as one long array of n_π blocks of b elements each, indexed through the *blockat* function. The first 2σ indices refer to blocks in T , the other indices to blocks overlaying A . The *blockat*(i) function returns a pointer to

¹ see § 3 for discussion on the selection of b

the beginning of the block at index i .

$$\text{blockat}(i) = \begin{cases} \text{address of } T[ib] & \text{if } 0 \leq i < 2\sigma \\ \text{address of } A[(i - 2\sigma)b] & \text{if } 2\sigma \leq i < n_\pi \end{cases} \quad (1)$$

The $n \bmod b$ elements at the end of A are copied into a scratch block at the start of the algorithm. They are unused until the contents of T are copied back to A at the end of the algorithm. The array π describes a permutation of these n_π blocks, such that $\pi[i]$ holds the index of the block at *logical index* i . Fig. 1 shows an example of this data structure for strings of $n_t = 4$ characters on initialisation.

Each block is either *allocated* or *unallocated*. An allocated block is either *full*, meaning that all of its elements hold data, or *partial*, meaning that some of its elements hold data. Unallocated blocks do not hold any data and are ready for reuse. At the first σ logical indices, a *head start* of σ unallocated blocks is found. They are followed by allocated blocks at logical indices $\sigma \leq i < f$, and finally unallocated blocks are logical indices $f \leq i < n_\pi$.

Array P tracks up to σ partial blocks. It holds in order of logical index partial blocks (i, l) described by logical index i and block length $0 \leq l < b$. Of each such partial block, the first l elements are used, and the other $b - l$ elements are unused. To simplify the implementation, the block at logical index $f - 1$ is always a partial block.

■ **Algorithm 3** finding the size of the next block

```

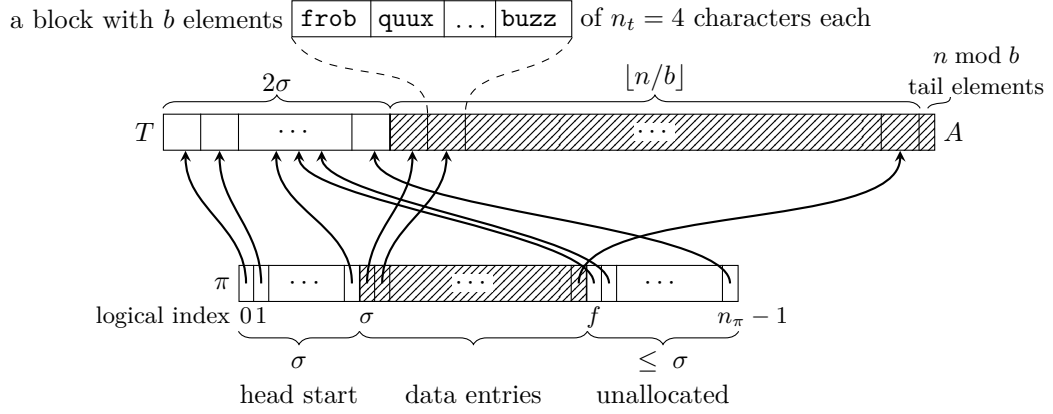
1 function SIZEOFNEXTBLOCK( $i, i_P$ )
2   ( $i_P, l_{i_P}$ )  $\leftarrow P[i_P]$ 
3   if  $\pi[i] = i_P$  then                                     ▷ is  $\pi[i]$  a partial block?
4      $i_P \leftarrow i_P + 1$ 
5     return  $l_{i_P}$ 
6   else return  $b$ 
7   end if
8 end function

```

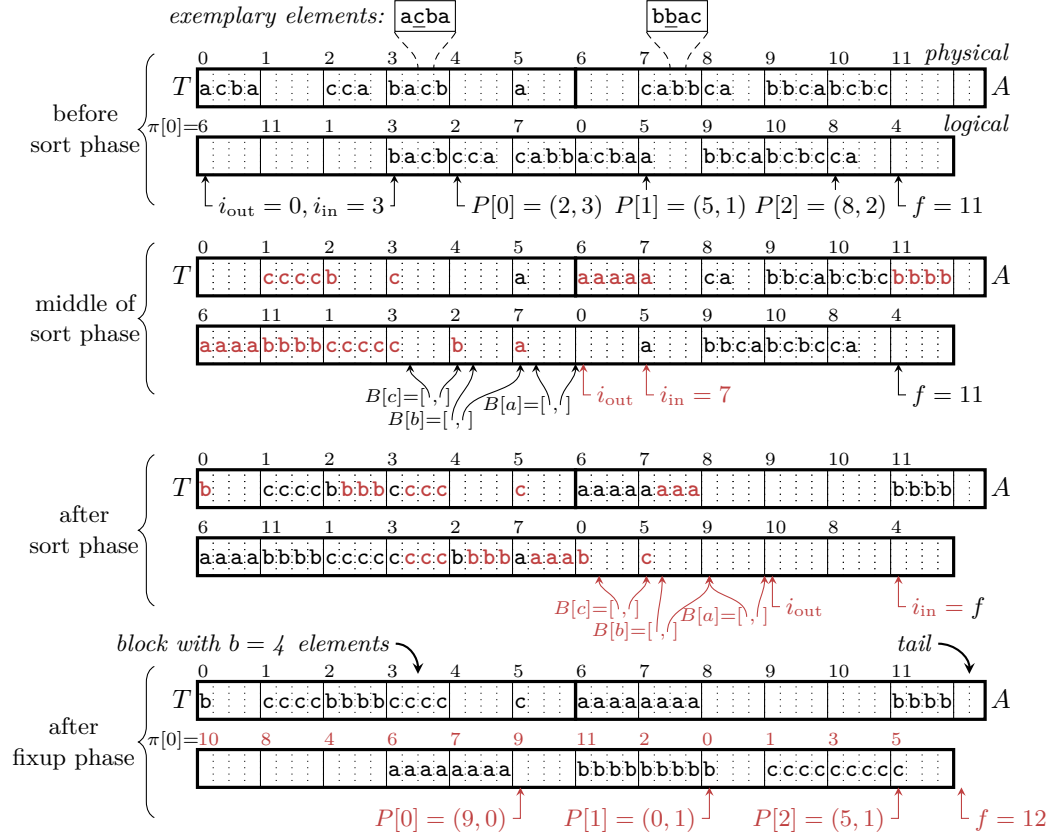
Concatenated in the order given by π , the contents of full and partial blocks hold the elements to be sorted in the order induced by the rounds of sorting performed so far. We can iterate over the allocated blocks in logical order, discovering the length of each block by traversing P at the same time. This idea is wrapped in the function `SIZEOFNEXTBLOCK( $i, i_P$ )` which gives the length of the block at logical index i , assuming the next partial block is tracked in $P[i_P]$. If the block is partial, i_P is updated to track the next partial block.

In summary, the state of the Radsort algorithm is tracked in the variables given below. Additional variables are needed within individual algorithm steps and are explained there.

A	input array	array of strings
n	input length	$n = A $
σ	alphabet size	$\sigma = \Sigma $
n_t	key length	integer
b	block size	integer
n_π	block count	$n_\pi = \lfloor n/b \rfloor + 2\sigma$
T	scratch buffer	array of $2\sigma b$ strings
f	fill level	integer
π	permutation	array of n_π integers
P	partial blocks	array of σ pairs of integers



■ **Figure 1** Radsort's data structures at initialisation



■ **Figure 2** Algorithm state during the second round of sorting an array of strings with $n = 26$, $n_t = 4$, $b = 4$, and $\Sigma = \{a, b, c\}$. Changes highlighted in red. Unallocated cells are blanked out.

2.2 Initialisation

The variables are initially set up (cf. Fig. 1) such that the contents of A are represented by the blocks at logical indices σ to $\sigma + \lceil n/b \rceil$. The final $n \bmod b$ elements of A are copied to block σ . This establishes the *data invariant* of the state variables, shown below. Each round of sorting assumes that this invariant holds initially and reestablishes it at conclusion.

1. π is a permutation of $0 \dots n_\pi - 1$
2. P tracks the partial blocks in ascending order of logical index
3. the block at logical index $f - 1$ is a partial block
4. logical indices 0 to $\sigma - 1$ and f to $n_\pi - 1$ refer to unallocated blocks
5. logical indices σ to $f - 1$ refer to allocated blocks, the concatenation of whose used elements holds the input permuted according to the current progress of the algorithm

■ **Algorithm 4** initialisation of variables

```

1 procedure SETUP( $A, n$ )
2    $f \leftarrow \sigma + \lfloor n/b \rfloor + 1$ 
3    $\pi[0 \dots \sigma - 1] \leftarrow 0, \dots, \sigma - 1$  ▷ assign head start
4    $\pi[\sigma \dots f - 2] \leftarrow 2\sigma, \dots, \sigma + f - 2$  ▷ assign contents of  $A$ 
5    $\pi[f - 1 \dots n_\pi - 1] \leftarrow \sigma, \dots, 2\sigma - 1$  ▷ assign remaining blocks
6    $T[0 \dots 2\sigma b - 1] \leftarrow 0$  ▷ filly  $T$  with dummy values (optional)
7    $\text{blockat}(\sigma)[0 \dots (n \bmod b) - 1] \leftarrow A[b\lfloor n/b \rfloor \dots n - 1]$  ▷ copy tail to scratch block
8    $P[0] \leftarrow (f - 1, n \bmod b)$  ▷ track input tail as partial block
9    $P[1 \dots \sigma - 1] \leftarrow (n_\pi, 0)$  ▷ fill  $P$  with dummy values (optional)
10 end procedure

```

2.3 Sorting

Each round of sorting (cf. Fig. 2) sorts the elements of A stably according to some key position t . We write $\text{key}(A[i], t) \in \Sigma$ for key position t of element $A[i]$. First, the *sort phase* sorts elements into buckets according to the value of their key. Then, the *fixup phase* computes new π and P based on the results of the sort phase, restoring the data invariant. The sort and fixup phases require several additional variables that can be discarded afterwards:

B	bucket allocation	array of σ pairs of pointers
C	block counts	array of σ integers
S	bucket starts	array of σ integers
U	block usage	array of n_π characters
π'	new permutation	array of n_π integers
i_{in}	next input block	integer
i_{out}	next output block	integer
i_P	next partial block	integer

2.3.1 Sort phase

The sort phase allocates one block for each bucket in the bucket array B . This array holds for each bucket c a pair of pointers $(p_{\text{next}}, p_{\text{end}})$ pointing to the next free element, and the end of the block. It then traverses the allocated blocks by i_{in} in logical order, and for each

block sorts its elements into the correct output block. If an output block is full, a new block is drawn from the next logical index i_{out} using the `NEWBUCKETBLOCK( $c$ )` procedure. The bucket each output block is used for is tracked in the usage array U , and the number of blocks assigned to each of the buckets is tracked in C .

As the sort proceeds through the input array, consumed blocks of input are reassigned into output blocks. It is critical that blocks of input are fully consumed before they are reassigned into output blocks, lest elements are overwritten before they can be sorted. This is ensured through i_{in} having a *head start* of σ unallocated blocks:

► **Lemma 1.** *When a new output block needs to be assigned, $i_{\text{out}} < i_{\text{in}}$.*

Proof. Let j be the number of elements processed so far. The input comprises of blocks holding at most b elements, so at least $\lfloor j/b \rfloor$ blocks of input have been processed so far, hence $\sigma + \lfloor j/b \rfloor \leq i_{\text{in}}$. As for output blocks, a new one is assigned whenever the current block of some bucket is full. Thus there are now $\sigma - 1$ partial output blocks (one for each bucket, except for the bucket we need to assign a new output block to) as well as up to $\lfloor j/b \rfloor$ full output blocks. Hence $i_{\text{out}} \leq \sigma - 1 + \lfloor j/b \rfloor$. Joining these two together gives

$$i_{\text{out}} \leq \sigma - 1 + \lfloor j/b \rfloor < \sigma + \lfloor j/b \rfloor \leq i_{\text{in}} \quad (2)$$

as required. ◀

■ **Algorithm 5** sort phase

```

1 procedure NEWBUCKETBLOCK( $c$ )                                ▷ allocate a new bucket block for key  $c$ 
2    $p_{\text{next}} \leftarrow \text{blockat}(\pi[i_{\text{out}}])$ 
3    $B[c] \leftarrow (p_{\text{next}}, p_{\text{next}} + b)$ 
4    $C[c] \leftarrow C[c] + 1$ 
5    $U[i_{\text{out}}] \leftarrow c$ 
6    $i_{\text{out}} \leftarrow i_{\text{out}} + 1$ 
7 end procedure
8 procedure SORTPHASE( $t$ )
9    $C[0 \dots \sigma - 1] \leftarrow i_P \leftarrow i_{\text{out}} \leftarrow 0$                                 ▷ initialise variables
10  for  $c \in \Sigma$  do NEWBUCKETBLOCK( $c$ )                                ▷ assign initial output blocks
11  for  $i_{\text{in}} \leftarrow \sigma \dots f - 1$  do
12     $p_{\text{in}} \leftarrow \text{blockat}(\pi[i_{\text{in}}])$                                 ▷ get next block of input
13     $l \leftarrow \text{SIZEOFNEXTBLOCK}(i_{\text{in}}, i_P)$ 
14    for  $j \leftarrow 0 \dots l - 1$  do
15       $c \leftarrow \text{key}(p_{\text{in}}[j], t)$                                 ▷ find bucket of current element
16       $(p_{\text{next}}, p_{\text{end}}) \leftarrow B[c]$ 
17      copy  $p_{\text{in}}[j]$  to memory at  $p_{\text{next}}$ 
18      if  $p_{\text{next}} + 1 = p_{\text{end}}$  then                                ▷ bucket full?
19        NEWBUCKETBLOCK( $c$ )
20      else
21         $B[c] \leftarrow (p_{\text{next}} + 1, p_{\text{end}})$ 
22      end if
23    end for
24  end for
25 end procedure

```

2.3.2 Fixup phase

After the sort phase, the blocks at logical index 0 to i_{out} hold the sorted buckets in some interleaving, with up to b partial blocks described by B . The fixup phase computes a new permutation π' that starts with a head start of σ unallocated blocks, followed by the deinterleaved buckets in order of keys and finally any remaining unallocated blocks. We are guaranteed to always find σ unallocated blocks for the head start:

► **Lemma 2.** *After the sort phase, there are at least σ unallocated blocks.*

Proof. A full block holds b elements. As there are n elements in total, there are at most $\lfloor n/b \rfloor$ full output blocks. Furthermore, there are σ partial blocks tracked in B . As each block after sorting is either used as an output block or unallocated, and as there are n_π blocks in total, we find for the number of unallocated blocks $n_{\text{unallocated}}$

$$n_{\text{unallocated}} \geq n_\pi - \sigma - \lfloor n/b \rfloor = \sigma \quad (3)$$

as required. ◀

The buckets are deinterleaved by first computing the starting indices $S[0 \dots \sigma - 1]$ of the buckets and then by shuffling the blocks according to the usages U into π' . Meanwhile, each allocated block is compared with the pointers in B to populate P with the whereabouts of the new partial blocks. As there is one partial block for each bucket, each of the σ elements of P ends up being initialised. Finally, π is set to π' to restore the data invariant.

■ Algorithm 6 fixup phase

```

1 procedure FIXUPPHASE
2    $S[0] \leftarrow \sigma$ 
3   for  $i \leftarrow 1 \dots \sigma - 1$  do  $S[i] \leftarrow S[i - 1] + C[i - 1]$            ▷ prefix sum over  $C$  plus  $\sigma$ 
4    $\pi'[0 \dots \sigma - 1] \leftarrow \pi[i_{\text{out}} \dots i_{\text{out}} + \sigma - 1]$            ▷ assign head start
5   for  $i \leftarrow 0 \dots i_{\text{out}} - 1$  do                                     ▷ shuffle used blocks into order of buckets
6      $c \leftarrow U[i]$ 
7      $\pi'[S[c]] \leftarrow \pi[i]$ 
8      $(p_{\text{next}}, p_{\text{end}}) \leftarrow B[c]$ 
9     if  $p_{\text{end}} = \text{blockat}(\pi[i]) + b$  then                               ▷ is  $\pi[i]$  a partial block?
10       $P[c] \leftarrow (S[c], p_{\text{next}} - \text{blockat}(\pi[i]))$ 
11    end if
12     $S[c] \leftarrow S[c] + 1$ 
13  end for
14   $\pi'[i_{\text{out}} + \sigma \dots n_\pi - 1] \leftarrow \pi[i_{\text{out}} + \sigma \dots n_\pi - 1]$    ▷ assign remaining blocks
15   $f \leftarrow i_{\text{out}} + \sigma$ 
16   $\pi \leftarrow \pi'$ 
17 end procedure

```

2.4 Finalisation

Following some rounds of sorting, the used elements of the blocks at logical indices σ to $f - 1$ hold the elements of the input in sorted order. To wrap things up, we need to shuffle this collection of logically ordered blocks back into A , preserving the ordering.

■ **Algorithm 7** restoration of the contents of A

```

1 procedure SWAP( $i_1, j_1, i_2, j_2$ )
2    $\pi[i_1], \pi[i_2] \leftarrow j_2, j_1$ 
3    $\pi^{-1}[j_1], \pi^{-1}[j_2] \leftarrow i_2, i_1$ 
4    $i_1, i_2 \leftarrow i_2, i_1$ 
5 end procedure
6 procedure FINALIZE
7    $k \leftarrow i_P \leftarrow 0$ 
8    $i_{\text{free}}, j_{\text{free}} \leftarrow 0, \pi[0]$   $\triangleright$  guaranteed unallocated as per invariant
9   for  $i \leftarrow 0 \dots n_\pi - 1$  do  $\pi^{-1}[\pi[i]] \leftarrow i$   $\triangleright$  compute inverse permutation
10  for  $i_{\text{in}} \leftarrow \sigma \dots \sigma + \lfloor n/b \rfloor - 1$  do  $\triangleright$  pull first  $\lfloor n/b \rfloor$  blocks
11     $j_{\text{in}} \leftarrow \pi[i_{\text{in}}]$ 
12     $j_{\text{out}}, i_{\text{out}} \leftarrow i_{\text{in}} - \sigma, \pi^{-1}[i_{\text{in}} - \sigma]$ 
13    if  $i_{\text{in}} < i_{\text{out}} < f$  then  $\triangleright j_{\text{out}}$  not the right block, but allocated?
14      copy block at  $j_{\text{out}}$  to block at  $j_{\text{free}}$   $\triangleright$  push block at  $j_{\text{out}}$  away
15      SWAP( $i_{\text{out}}, j_{\text{out}}, i_{\text{free}}, j_{\text{free}}$ )  $\triangleright$  track swapping of  $i_{\text{free}}$  and  $i_{\text{out}}$ 
16    end if
17     $l \leftarrow \text{SIZEOFNEXTBLOCK}(i_{\text{in}}, i_P)$ 
18     $A[k \dots k + l - 1] \leftarrow \text{blockat}(j_{\text{in}})[0 \dots l - 1]$   $\triangleright$  pull block from  $j_{\text{in}}$ 
19     $k \leftarrow k + l$ 
20    SWAP( $i_{\text{out}}, j_{\text{out}}, i_{\text{in}}, j_{\text{in}}$ )  $\triangleright$  track swapping of  $i_{\text{in}}$  and  $i_{\text{out}}$ 
21    if  $i_{\text{in}} \neq i_{\text{out}}$  then  $\triangleright$  did we pull from elsewhere?
22       $i_{\text{free}}, j_{\text{free}} \leftarrow i_{\text{in}}, j_{\text{in}}$   $\triangleright$  if yes, there is now a new free spot
23    end if
24  end for
25  for  $i_{\text{in}} \leftarrow \sigma + \lfloor n/b \rfloor \dots f - 1$  do  $\triangleright$  pull any remaining blocks (all in  $T$ )
26     $j_{\text{in}} \leftarrow \pi[i_{\text{in}}]$ 
27     $l \leftarrow \text{SIZEOFNEXTBLOCK}(i_{\text{in}}, i_P)$ 
28     $A[k \dots k + l - 1] \leftarrow \text{blockat}(j_{\text{in}})[0 \dots l - 1]$   $\triangleright$  pull block from  $j_{\text{in}}$ 
29     $k \leftarrow k + l$ 
30  end for
31 end procedure

```

This is achieved by shuffling the used blocks around such that they are found at indices $2\sigma \dots 2\sigma + f - 1$. When the blocks are moved into place, their elements are actually moved right behind the elements of the previous block, ignoring gaps of unused elements at the end of partial blocks, and leaving the elements in their final locations. After $\lfloor n/b \rfloor$ blocks have been shuffled to indices 2σ to $n_\pi - 1$, all remaining blocks must be located in T . As an optimisation, we can directly copy them to their final positions without moving other data away, as there is no risk of other blocks being in their way.

In this step, pairs of indices i_o, j_o refer to the logical and physical indices of the same block. These are related through the identity

$$j_o = \pi[i_o], \quad i_o = \pi^{-1}[j_o]. \quad (4)$$

The following extra variables are used:

π^{-1}	inverse permutation	array of n_π integers
$i_{\text{in}}, j_{\text{in}}$	block holding next data chunk	integers
$i_{\text{out}}, j_{\text{out}}$	where next block goes	integers
$i_{\text{free}}, j_{\text{free}}$	some unallocated block	integers
i_P	next partial block	integer
k	no. of elements written into A	integer

To help us carry out these shuffles, we first compute an inverse permutation π^{-1} such that $\pi \circ \pi^{-1} = (0, \dots, n_\pi - 1)$. The assignment of two blocks j_1 and j_2 can be swapped with the $\text{SWAP}(i_1, j_1, i_2, j_2)$ function, while moving their contents in a separate step.

For each block index $2\sigma \leq j < n_\pi$, we distinguish three cases: (a) the right block is already at index j , in which case we just move it ahead to fill the gap, (b) the block is unallocated, in which case we find the right block at $\pi^{-1}[j]$ and *pull* it to its final position, or (c) there is some other allocated block at index j , in which case we *push* it to some free block and reduce to case (b). The variable j_{free} tracks a spot to push an empty block to. whenever this spot is occupied, we are going to subsequently pull a block from somewhere else, restoring the unallocated spot.

3 Complexity Analysis

For analysis, we assume array element size and alphabet size σ to be constants and $b \in \mathcal{O}(n)$.

Runtime The $\text{SETUP}(A, n)$ procedure from Alg. 4 initialises arrays of $n_\pi \in \mathcal{O}(n/b)$ and b elements respectively for a total runtime of $\mathcal{O}(n/b + b) \subset \mathcal{O}(n)$. The $\text{SORTPHASE}(t)$ procedure from Alg. 5 executes one inner loop iteration for each of the n array elements for a total runtime of $\mathcal{O}(n)$. The FIXUPPHASE procedure from Alg. 6 writes to each element of π' once, for a total runtime of $\mathcal{O}(n/b)$. Lastly, the FINALIZE procedure from Alg. 7 traverses π , copying $\lfloor n/b \rfloor$ blocks at most twice, while copying the remaining $f - \lfloor n/b \rfloor - \sigma \leq \sigma$ blocks at most once for a total runtime of $\mathcal{O}(b(n/b + \sigma)) \subset \mathcal{O}(n)$. For the full sort given in Alg. 2, the $\text{SORTPHASE}(t)$ and FIXUPPHASE procedures are called once for each key position, raising the total runtime to $\mathcal{O}(n_t n)$, which matches the conventional LSD radix sort.

Space In addition to the input stored in A , array T occupies $\mathcal{O}(b)$ bytes of storage, arrays U , π , π' , and π^{-1} occupy $\mathcal{O}(n/b)$ bytes of storage, and the other variables are of constant size, for a total extra storage requirement of $\mathcal{O}(b + n/b)$. This requirement can be minimised by choosing some $b \in \Theta(\sqrt{n})$, giving just $\mathcal{O}(\sqrt{n})$ bytes of extra storage.

4 Implementation Notes and Optimisations

We have produced a variety of Radsort implementations in the C language. We make our code freely available.² Implementations in other languages should be straightforward; in languages that do not provide pointers or pointer arithmetic, the B array can be altered to use the same data structure as the P array, although at a loss of performance.

Using the pointer-based data structure of the B array for the P array is not advisable: While FIXUPPHASE (Alg. 6) is simplified by not needing to translate B into P , performance

² See <https://github.com/clausecker/radsort>.

of $\text{SORTPHASE}(t)$ (Alg. 5) is reduced due to the longer dependency chain when checking if the next input block is partial. Additionally, the FINALIZE procedure (Alg. 7) needs partial blocks to be tracked by their logical indices, mandating that the data structure be translated at least once.

A radix of $\sigma = 256$ appears to be optimal for sorting key-value pairs of integers, though the best choice depends on cache and element size, as well as key type and distribution and must be determined empirically. The basic tradeoff is that larger radices allow for less rounds of sorting, but require more cache to track the buckets, eventually escalating into higher levels of the cache hierarchy at great performance cost.

For a type-generic implementation, the $\text{SORTPHASE}(t)$ procedure should be monomorphised, ideally for each key position. In the common case of integer keys split into byte-sized digits ($\sigma = 256$), retrieving the byte at position t of the key by means of shifts and masks performs better than direct loading of the key byte from memory, as the latter generally incurs an extra memory operation, while the former shifts the load to arithmetic execution units that are otherwise underutilised.

While a block size of order $b \in \Theta(\sqrt{n})$ minimises space overhead, choosing a fixed block size simplifies the implementation. The space overhead then becomes some fixed fraction of the input size, which is often acceptable. For example, we used $b = 512$ in our experiments. At an element size of 8 B, this gives a fixed overhead of 2 MiB for the T array, 8 KiB for arrays B , C , P , and S together, and 9 B per block of input (i.e. 0.22% the input data set) to store π , π' , π^{-1} , and U , reusing π' for π^{-1} .

Like with a conventional LSD radix sort, performance of Radsort is memory bound and eliminating just a single load or store from the inner loop of $\text{SORTPHASE}(t)$ can improve performance dramatically, as those memory accesses compete for resources with the write in Alg. 5, L. 17, the bottleneck of the algorithm. One technique to eliminate such a load is discussed in § 4.2, and gives a 5–50% speedup depending on array size and microarchitecture.

4.1 Avoiding Finalisation

In some use cases, it suffices to iterate over the sorted data. In such cases, the FINALIZE procedure can be skipped, employing instead the $\text{SIZEOFNEXTBLOCK}(i, i_P)$ function from Alg. 3 to iterate over the array elements in permuted representation as in Alg. 5 (see also § 2.1).

Using such an iterator pattern, the dataset can be modified in place, and even be truncated (cf. Alg. 8) without disturbing the Radsort algorithm state. This enables algorithm designs where the same dataset is repeatedly sorted by different keys, and then modified, without having to finalise the sort or recreate the algorithm state for every sort.

■ **Algorithm 8** truncate the dataset such that logical block i , element j is the final datum

```

procedure TRUNCATE( $i, j$ )
   $f \leftarrow i + 1$                                  $\triangleright$  mark logical block  $i$  as the final block
  for  $i_P \leftarrow 0 \dots \sigma - 1$  do            $\triangleright$  skip partial blocks before logical block  $i$ 
     $(i_{i_P}, l_{i_P}) \leftarrow P[i_P]$ 
    if  $i \leq i_{i_P}$  then
       $P[i_P] \leftarrow (i, j + 1)$                  $\triangleright$  logical block  $i$  ends at element  $j$ 
    return
  end if
end for
end procedure

```

4.2 Faster End-of-Block Checking

The `SortPhase`(t) procedure of Alg. 5 tracks output blocks through pairs $(p_{\text{next}}, p_{\text{end}})$ pointing to the next free element and the end of the block, respectively. It is beneficial to reduce this pair to just one pointer, as to eliminate a load of the second pointer in the hot loop, and to reduce cache pressure.

In a simplified model, where each element is one byte in size and the allocation of all data structures including A can be controlled, this can be achieved by choosing some block size $b = 2^q$ and ensuring both A and T are aligned to a multiple of b . Then, no p_{end} is needed, as a pointer into some block of A or T points to a block boundary iff it is aligned to a multiple of b bytes—which can easily be checked with a bitwise-and on the pointer.³

In real applications, we usually cannot control the alignment of A and neither is the element size guaranteed to be some nice number. However, with some modifications, the same idea still works: let the element size be $2^r e$, with e odd and the block size again be $b = 2^q$. Then in an array of such elements of no particular alignment, every b elements there is an element with address p_0 such that

$$p_0 \gg r \equiv 0 \pmod{b}, \quad (5)$$

where $\gg$ denotes a bitwise logical right-shift. This condition is just as easy to check using a bitwise-and on the pointer and allows us to find block boundaries if the first element of each block satisfies Eq. 5. Furthermore, for the address of an arbitrary array element p , we can find the offset $0 \leq o < b$ of such an element from p as

$$o = -(p \gg r)e^{-1} \pmod{b} \quad (6)$$

where e^{-1} is the modular inverse of e modulo b . Due to b being a power of two, e^{-1} is easy to compute [4]. This gives the following modifications: allocate $2\sigma + 1$ blocks⁴ for T aligned to a multiple of 2^{qr} , ensuring that its block address satisfy Eq. 5. Compute the offset o of the first element of A satisfying Eq. 5 using Eq. 6. The $\text{blockat}(i)$ function is adjusted to

$$\text{blockat}(i) = \begin{cases} \text{address of } T[i] & \text{if } 0 \leq i < 2\sigma + 1 \\ \text{address of } A[(i - 2\sigma - 1)b + o] & \text{if } 2\sigma + 1 \leq i < n_\pi \end{cases} \quad (7)$$

and $n_\pi = \lfloor (n - o)/b \rfloor + 2\sigma + 1$. Transfer the first o elements to a scratch block and adjust Alg. 4 such that this scratch block (the head) ends up at logical block σ , followed by the blocks overlaying A , following the tail. Various other steps of the algorithm must also be adjusted to account for the extra scratch block. The end-of-block check then becomes a check for the condition of Eq. 5, which is realised by masking the pointer with a bitmask of the form $(b - 1) \ll r$ and checking if the result is zero. The p_{end} pointers no longer need to be read and can be eliminated entirely.

This approach yielded a 5–50 % speedup depending on array size and microarchitecture. It should be considered if the programming language and environment permit it.

³ The $p_{\text{end}} = \text{blockat}(\pi[i]) + b$ check of Alg. 6, L. 9 must then be realised with a range check like $\text{blockat}(\pi[i]) \leq p_{\text{next}} < \text{blockat}(\pi[i]) + b$. This is not strictly legal in the C23 language [6, § 6.5.8 ¶ 6], (pointers may only be compared for ordering if they point into the same array, but p_{next} may point into either A or T) but unproblematic in practice.

⁴ An extra scratch block is needed as the total number of elements shunted to head and tail may be up to $2b - 2$, possibly exceeding a whole block of elements.

4.3 Parallel Operation

The algorithm can be parallelised with some changes to the data structures. Before each sort phase, we distribute the blocks of the input array into n_t roughly evenly sized chunks, for sorting with n_t threads. Each chunk gets its own head start of σ blocks, requiring a T array of $2\sigma n_t$ scratch blocks in total. The sort phase then proceeds in parallel with each thread sorting its chunk with its own B array. The fixup phase is sequential and must interleave the individual thread's B arrays into one global P array of $n_t\sigma$ entries each.

The finalisation procedure is more troublesome to parallelise. It can be left as a sequential operation, as it only contributes a small amount of the total run time, or it can be implemented by first shuffling all the blocks into order using a parallel permutation algorithm [3], and then moving the array elements to eliminate gaps caused by partial blocks.

As Radsort is memory bound and exhibits only moderate cache locality, a performance ceiling quickly is reached as the available memory channels are saturated. See § 5.2 for more discussion. It may be of interest to use an initial round of sorting on the most significant key position to split the input into buckets that can be parcelled into threads, each of which processes the remaining key positions of its bucket(s) from the least-significant digit sequentially. This reduces the working set of each thread from n to n/n_t elements on average, improving cache efficacy and reducing NUMA effects.

5 Evaluation

We evaluated the performance of Radsort in comparison to a classic out-of-place LSD radix sort on a variety of systems. Five algorithm variants are compared in total:

generic A generic out-of-place LSD radix sort (Alg. 1) with output prefetching [2]. An initial pass takes histograms of the 4 key bytes, followed by 4 rounds of sorting.

swc An out-of-place LSD radix sort implemented with Wassenberg's software-defined write-combining [8] with a block size of 512 B.

radsort Single-threaded Radsort (Alg. 2) as described in § 2 with none of the improvements mentioned in § 4 using a fixed block size of $b = 512$ (i.e. 4 KiB).

bitmanip Single-threaded Radsort implemented using the bit-manipulation based end-of-block checking described in § 4.2, otherwise the same as *radsort*.

parallel Multi-threaded Radsort using 4 threads by default. Otherwise the same as *radsort*.

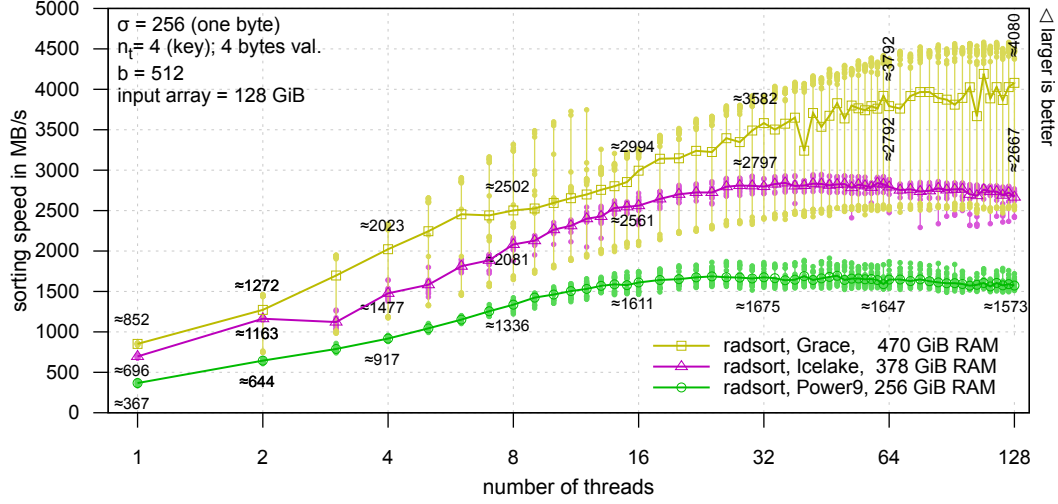
5.1 Setup

All benchmarks sort pairs of 32-bit keys and 32-bit values in $n_t = 4$ rounds by each key byte in turn ($\sigma = 256$). The key values are uniformly distributed over the range $0 \dots 2^{32} - 1$, as generated by a xorshift RNG [5]. While we have evaluated radsort on a variety of platforms, we have selected benchmark results as measured on the following machines for this paper:

	power	icelake	grace
<i>architecture</i>	powerpc64le	amd64	aarch64
<i>CPU</i>	IBM POWER9	Intel Xeon Gold 6338	ARM Neoverse V2
<i>sockets/cores/threads</i>	2 / 16 / 4	2 / 32 / 2	2 / 72 / 1
<i>threads total</i>	128	128	144
<i>clock speed</i>	2.9 GHz	2.0 GHz	3.4 GHz
<i>L1D/L2 cache per core</i>	32 KiB / 512 KiB	48 KiB / 1280 KiB	64 KiB / 1024 KiB
<i>L3 cache</i>	10 MiB per 2 cores	48 MiB per socket	114 MiB per socket

Two sets of benchmarks were performed: in the *size* sets (Fig. 4), all algorithms were measured on arrays with 2^n and $3 \times 2^{n-1}$ elements with array sizes from 2 to as many elements as the memory fits. In the *threads* sets (Fig. 3), the *parallel* implementation was measured with an array size of 128 GiB (i. e. 2^{34} elements) and thread counts from 1 to 128.

20 runs of each benchmark were performed. In the plots, dots show the individual runs, with a line drawn through the average. For the size sets, execution was pinned to a single socket to avoid NUMA effects as much as possible.



■ **Figure 3** Sorting speed compared on a 128 GiB array with thread count from 1 to 128.

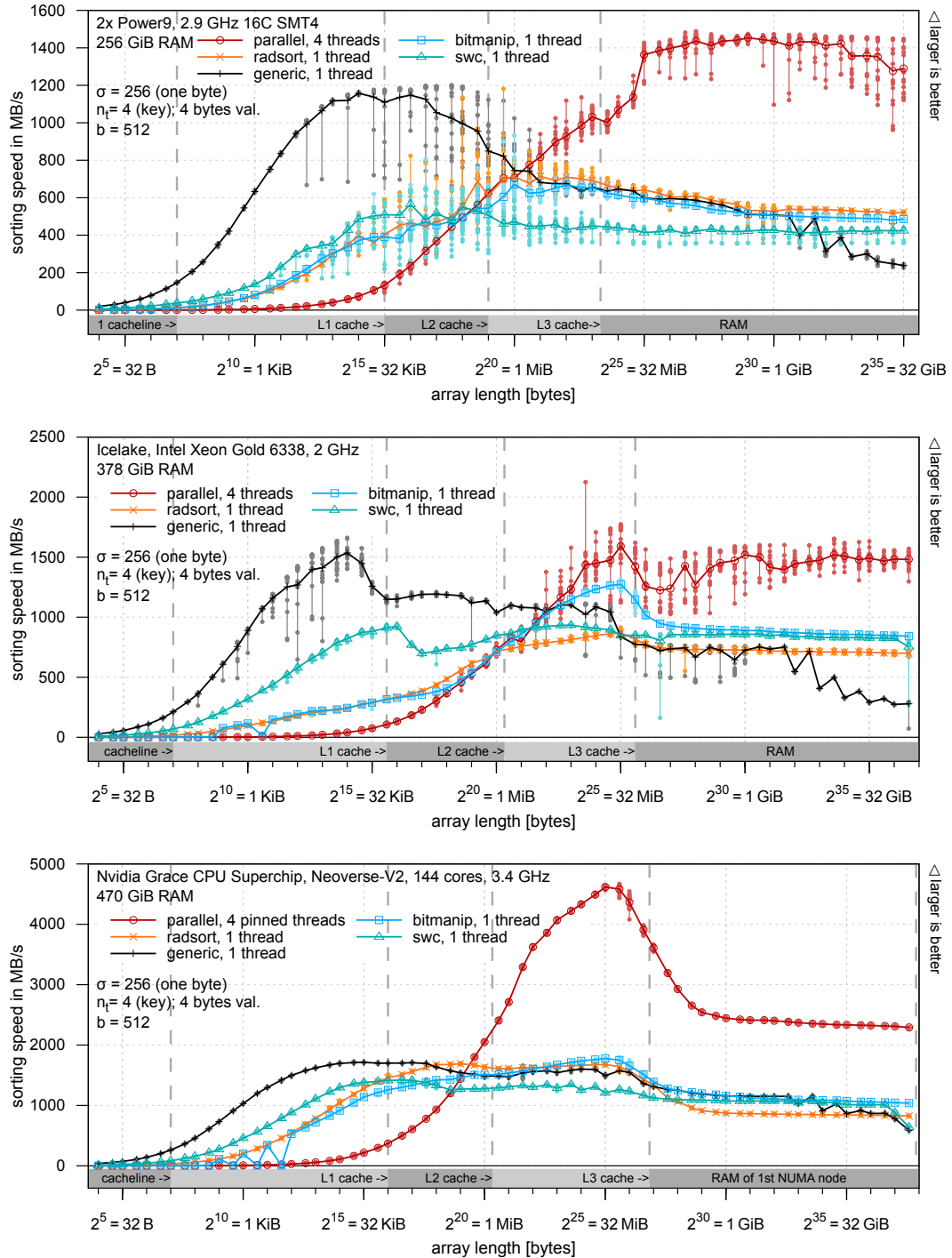
5.2 Results

For small arrays up to around twice the L2 cache size, the *generic* implementation is the clear winner. As cache misses are mostly absent at these array sizes, the overhead of Radsort cannot be outweighed by its cache-locality benefits. As input size exceeds this limit, Radsort and its variants quickly outperform the generic implementation. Implementing bit-manipulation based end-of-block checking yields a speedup of 5–50 % on all systems tested (including others not shown here), except for the *power* system where it is slightly slower.

As expected, the *swc* variant performs very consistently over the whole range of array lengths, as it minimises cache effects through manual write combining. This consistency is maintained even as the array size approaches a significant chunk of the main memory, while the generic implementation drops to around half of its performance, matching the results observed by Wassenberg et al. [8]. The *bitmanip* variant exceeds this performance on all systems for array sizes above the L2 cache size, showing that its timely reuse of input blocks for output blocks avoid read-to-own transfers just as effectively as *swc*.

Whenever it is worth using Radsort over an out-of-place radix sort, the parallel variant outperforms the scalar variants, though the advantages quickly diminish (cf. Fig. 3), topping out around 24 threads on the *icelake* machine and around 48 threads on the *power* machine. NUMA heavily impacts the *grace* machine, causing high measurement variance based on thread placement. Performance peaks here at around 72 threads.

In summary, it seems advantageous to use *generic* for short arrays, and to switch to *bitmanip* or *parallel* as input exceeds some empirically determined threshold. This threshold



■ **Figure 4** Sorting speeds on three machines with arrays of 8 B to 96 GiB (192 GiB bottom).

could be set to the size of T , allowing for the reuse of T as the output array A' of Alg. 1.

6 Related Work

Radsort is an attempt to adapt the ideas of IPS²RA [1] to LSD radix sort. While IPS²RA is designed to use $\mathcal{O}(1)$ extra memory, the author found this difficult to achieve under the LSD approach’s stability requirements. By raising the overhead to $\mathcal{O}(\sqrt{n})$, the permutation of blocks can be explicitly tracked in π , addressing these challenges. As a consequence, blocks only have to actually be moved around during finalisation, giving a significant speedup.

The result is similar to the “out-of-cache, in-place, list of blocks” partitioning described by Polychroniou et al. [7, § 3.2.3], but uses arrays over linked lists to track π , improving cache utilisation, and permitting a straightforward finalisation procedure.

A major performance limitation of radix sorts is the need for output buckets to be read into cache so that they can be written to (read for ownership), effectively halving the write bandwidth and causing cache misses. While CPUs provide write-combining buffers to avoid this problem, radix sorts have a fan-out that exceeds their number, rendering them ineffective. The performance impact can be reduced by prefetching the output buckets speculatively [2], or more thoroughly by manual software-defined write-combining [8].

Radsort solves the problem more elegantly: as input blocks are reused for output blocks after only a short delay, the output blocks are still hot in cache⁵ when written to, avoiding an extra read for ownership. Consequently, we have found prefetching to have little to no performance impact and software write-combining to not be needed.

7 Future Work

It is promising to adapt Radsort to the sorting of dissimilarly sized elements, such as lines of text or JSON records, directly, i. e. without resorting to sorting an array of pointers to variable-length records. Likewise, adapting the general approach to external sorting should be investigated.

During the research for this paper, the authors investigated whether Radsort’s data structures would work for an MSD (i. e. recursive) radix sort procedure. Keeping the data structures identical, we quickly run into problems: while each recursive iteration consumes one partial block from the bucket it recurses over, it produces up to σ new partial blocks, leading to an uncompetitive memory overhead. The authors found that this problem can be addressed by compacting the partial blocks into a sequence of dense blocks after each iteration, tracking the beginning and end of partial data within this sequence using an extra data structure. This reduces the extra space for partial blocks to one block per recursion level plus 2σ words to track the number of elements in the partial tail of each bucket, requiring $\mathcal{O}((\sigma + b)n_t)$ extra space. Due to time constraints, this idea was not explored further.

8 Conclusion

In Radsort, we provide a straightforward way to implement LSD radix sorting with just $\mathcal{O}(\sqrt{n})$ space overhead. Radsort readily parallelises to a moderate number of threads. The

⁵ As long as the cache fits up to σb elements that have been consumed but not yet reused as output blocks.

performance can be further improved using bit manipulation techniques, at the cost of a more complex implementation.

As a consequence of more effective cache utilisation, the performance of Radsort is competitive with a standard out-of-place LSD radix sort for arrays as small as 2 MiB, even when using only one thread. The read-to-own bottleneck on large datasets is avoided without the requirement of platform-specific techniques like software-defined write-combining.

References

- 1 Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. Engineering in-place (shared-memory) sorting algorithms. *ACM Trans. Parallel Comput.*, 9(1):1–62, 2022. doi:10.1145/3505286.
- 2 Travis Downs. Beating up on qsort, 2019. URL: <https://travisdowns.github.io/blog/2019/05/22/sorting.html>.
- 3 Torben Hagerup and Jörg Keller. Fast parallel permutation algorithms. *Parallel Process. Lett.*, 5(2):139–148, 1995. doi:10.1142/S0129626495000126.
- 4 Jeffrey Hurchalla. An improved integer modular multiplicative inverse (modulo 2^w), 2022. arXiv:2204.04342.
- 5 George Marsaglia. Random number generators. *J. Mod. Appl. Stat. Methods.*, 2(1):2–13, 2003. doi:10.22237/jmasm/1051747320.
- 6 JeanHeyd Meneide and Freek Wiedijk. Programming languages – C. Standard ISO/IEC 9899:2023, International Organization for Standardization, 2024.
- 7 Orestis Polychroniou and Kenneth A. Ross. A comprehensive study of main-memory partitioning and its application to large-scale comparison- and radix-sort. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, SIGMOD ’14, pages 755–766. Association for Computing Machinery, 2014. doi:10.1145/2588555.2610522.
- 8 Jan Wassenberg and Peter Sanders. Faster radix sort via virtual memory and write-combining, 2010. arXiv:1008.2849.